

# Documento de Justificación Técnica

## Taller 1: Java Backend Avanzado

Proyecto de E-Commerce

23 de mayo de 2026

### 1. Estrategia de Concurrencia

Para abordar la regla de negocio central que impide la sobreventa de productos al procesar múltiples órdenes simultáneas, se eligió implementar un **Control de Concurrencia Optimista (Optimistic Locking)**.

**Justificación:** En el contexto de un sistema de comercio electrónico común, la probabilidad de que dos o más personas intenten modificar el stock del *exactamente el mismo producto* en la misma fracción de segundo es relativamente baja (comparado con otros sistemas de alta contención). Por tanto, un bloqueo pesimista degradaría innecesariamente el rendimiento general del sistema al bloquear filas en la base de datos para lecturas recurrentes.

Dejamos que las transacciones sucedan de manera normal, pero validamos la modificación mediante un campo de versión (`@Version`). Si durante el transcurso de una compra, otro usuario se adelanta y completa una orden sobre el mismo ítem, la transacción del segundo usuario lanzará un error (al no coincidir las versiones), cancelando la orden de manera segura y lanzando un código de error HTTP 409 (Conflict). Esto protege la integridad del inventario sin sacrificar la agilidad del sistema.

### 2. Estrategia de Autenticación y Seguridad

Se implementó un esquema de seguridad **Stateless basado en JSON Web Tokens (JWT)**, complementado con Control de Acceso Basado en Roles (RBAC).

**Justificación:** Al ser una API REST moderna, mantener el estado en el servidor (sesiones HTTP) limita la escalabilidad horizontal. JWT permite que el servidor valide la identidad y roles (ej. `ADMIN` vs `CLIENTE`) del usuario en cada petición sin necesidad de consultar la base de datos de manera constante para verificar la sesión, validando simplemente la firma y fecha de expiración del token. Además, las contraseñas se protegen utilizando un algoritmo de hash robusto antes de su persistencia.

### 3. Arquitectura Elegida

La solución se construyó sobre una **Arquitectura en Capas (Layered Architecture)**.

**Justificación:** Aunque existen patrones más desacoplados (como la Arquitectura Hexagonal o Clean Architecture), la arquitectura en capas fue la elección óptima para este proyecto por las siguientes razones:

- **Curva de aprendizaje y pragmatismo:** A lo largo del curso aún no se había profundizado en la implementación de Arquitectura Hexagonal en Java. Introducirla sin un dominio completo del patrón podría generar código sobre-diseñado o incorrecto.
- **Carga de trabajo (Single Developer):** Desarrollar un sistema con puertos y adaptadores estrictos implica una gran cantidad de mapeo, interfaces y abstracciones, lo cual es prohibitivo para un desarrollo individual bajo limitaciones de tiempo. Las capas ofrecen una separación de responsabilidades clara y suficiente para la complejidad actual del dominio.

## 4. Detalle de las Capas y Paquetes Específicos

El proyecto se dividió rigurosamente para garantizar un bajo acoplamiento, reflejado en la siguiente estructura de paquetes dentro de `src/main/java/org/java_avanzado/taller`:

- **domain:** Contiene la lógica y modelos de negocio puros (`model` y `enums`). Esta capa desconoce cualquier framework de persistencia o red.
- **persistence:** Encargada de la interacción con la base de datos PostgreSQL. Contiene las `entity` (mapeo JPA), los `repository` (interfaces de Spring Data JPA) y las `specification` para consultas dinámicas.
- **service:** Actúa como orquestador. Aplica las reglas de negocio transaccionales interactuando con los repositorios y modelos de dominio.
- **controller:** Responsable de la capa HTTP. Contiene los endpoints REST. Sus subpaquetes organizan la entrada/salida:
  - **dto:** Objetos de Transferencia de Datos divididos en `request` (para entrada de datos, creación y filtrado usando `@Data` y validaciones de Jakarta) y `response` (salida usando `Records`).
  - **advice:** Contiene el `GlobalExceptionHandler` para capturar errores de manera centralizada y retornar códigos HTTP limpios.
- **security:** Configura Spring Security e incluye la implementación del filtro para validación de tokens en el subpaquete `jwt`.
- **config:** Archivos de configuración general de la aplicación y Beans administrados (como configuraciones CORS o de base de datos extra).
- **exception:** Excepciones de dominio personalizadas.
- **utils:** Herramientas transversales:
  - **mapper:** Interfaces de `MapStruct` para transformar de `Entity` a `Model` a `DTO`, manteniendo las capas limpias y evitando que un `DTO` llegue al dominio.
  - **validators:** Lógica reutilizable de validación.

## 5. Cambios Realizados desde el Último Commit

Con base en el taller 2 de java , se identificaron cambios estructurales y funcionales relevantes en la solución.

### 1) Reestructuración del repositorio

- El proyecto pasó de una estructura monolítica en raíz a un esquema **multi-módulo Maven** con empaquetado pom en el padre.
- Se incorporaron tres módulos: `event-contracts`, `commerce-app` y `payments-service`.
- Se eliminó la base de código previa ubicada en `src/main` y `src/test` de la raíz, trasladando la lógica de negocio a `commerce-app`.

### 2) Evolución funcional: de flujo sincrónico a flujo dirigido por eventos

- Se centralizó la topología de mensajería en `event-contracts` mediante `EventTopology` y contratos de evento versionados (`PaymentRequestedEvent`, `PaymentSucceededEvent`, `PaymentFailedEvent`).
- `commerce-app` publica solicitudes de pago al completar checkout y consume resultados de pago para actualizar estado de órdenes (aprobación/rechazo y reposición de stock cuando aplica).
- `payments-service` procesa solicitudes consumiendo eventos de RabbitMQ, simula el resultado y publica respuesta de éxito o fallo.
- Se añadió endpoint de checkout asíncrono (respuesta HTTP 202) con identificador de correlación para trazabilidad de extremo a extremo.

### 3) Persistencia, idempotencia y pruebas

- Se agregaron mecanismos de idempotencia para resultados de pago procesados en `commerce-app` y para intentos en `payments-service`.
- Se extendieron pruebas de integración alrededor de la coreografía de pagos y la topología RabbitMQ (incluyendo soporte con Testcontainers).
- Se actualizó la configuración de infraestructura local en `docker-compose.yml` para ejecutar PostgreSQL y RabbitMQ.

## 6. Diagrama de Arquitectura

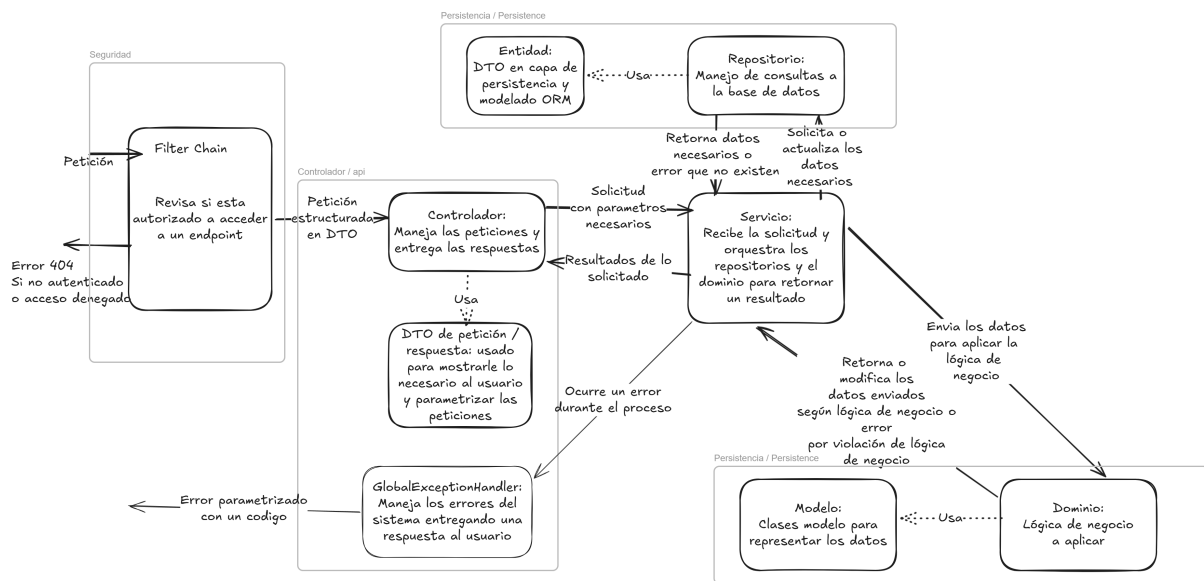


Figura 1: Diagrama de Arquitectura en Capas de la solución planteada